

GSoC21 W3: Kind, Characters, and Standards

Rohit Goswami · 2021-06-26 · gsoc21 · fortran · ramblings

Standard practice pragmatic approaches to `kind` for `dftatom`

Background

Serialized update for the [2021 Google Summer of Code](#) under the [fortran-lang organization](#), mentored by [Ondrej Certik](#).

Logistics

- Met with Ondrej on Tuesday
 - Went over my `kind` implementation
 - Merged older approved MRs
 - Worked on generating tests
- Talked about the test methodology in general
 - Most of the tests are better off in their integration form (discussed below)
 - Some aspects of the passes may be tested using the `doctest` setup
- Set an additional time to discuss the implementation of **assumed length** character declarations
 - These are not actually used in any `dftatom` routines but they are very common for utility functions
- Met with Ondrej on Thursday
 - Discussed repercussions of backends
 - Better, more explicit ASR rules can stem from not relying on the CPP backend
- Talked about the number of passes (SRC->AST->ASR->LLVM)
- Started working on getting the right thing happen when faced with `character(len=*)`

Overview

This week also saw an increase in community activities on the [Fortran discourse](#), since the J3 meeting is now underway and user polling [^fn:1] is in full swing.

New Merge Requests

Implement more `kind()` (997)

Added tests and code to the AST->ASR pass for `kind` calls relevant to `dftatom`

Draft: Implement **assumed length** (1000)

More of a trailer for the next week; also happens to be the 1000th MR (which is neat)

Freshly Assigned Issues

Some more Kind considerations (373)

More of a speculative issue about standards compliance, led to the more concrete and general [375](#)

Additional Tasks

Still unofficially planning to take a stab at [issue 350](#); regarding the main `lfortran.org` site.

Kinds

The standard allows for any constant to be used in a `kind` function call. Some commonly seen variants in the wild are:

```
integer, parameter :: dp=kind(0.d0), &      ! double precision
                        hp=selected_real_kind(15), & ! high precision
                        qp=selected_real_kind(32), & ! quadruple precision
                        sp = kind(0.)          ! single precision
```

Most commonly; `kind(0.d0)` is seen in the wild. Currently the `lfortran` ASR defines:

`ConstantInteger`, `ConstantReal`, `ConstantComplex`, `ConstantLogical` of which only `ConstantLogical` is implemented.

The idea is that `0.` is single precision; while `1._dp`, `1.d0` or `0.d0` are double precision. Essentially then the solution presented itself naturally; to check if `d` is present; assign double precision if true, otherwise stick to single precision. At the moment, `lfortran` considers either `4` for single or `8` for double.

More generally, it might be useful to make the single and double precision values managed by a pre-processor. For the other kinds of precision, some more thought is required.

At this stage, it compiles with the LLVM backend.

```
lfortran -v -c --show-asr --show-stacktrace lapack.f90
lfortran -v -c --backend=llvm --show-stacktrace lapack.f90
```

Tests

While implementing I used the simplest of debugging concepts, that of making manual changes and writing out results. However, no compiler can survive without rigorous unit tests, and `lfortran` is no different.

Fortran Integration Tests

So called because these are run by a `python` driver and store verbatim the `stdout` in files. Note that this form of testing, though convenient, does require the user to be certain of the test before writing it; due to the update process it is possible to store the wrong result and consistently get it wrong.

```
./run_tests.py -u # Update
```

Unit Tests for Passes

These are not favored, as any change in any of the passes causes a large amount of test metadata to be invalidated. There are some tests of this nature, and the framework used is `doctest`.

Towards Assumed Lengths

The first step towards implementing assumed lengths was to isolate the problem, which was done through the age-old comment recompile and test methodology. The offending function is a rather innocuous looking helper function:

```
function upcase(s) result(t)
! Returns string 's' in uppercase
character(*), intent(in) :: s
character(len(s)) :: t
integer :: i, diff
t = s; diff = ichar('A')-ichar('a')
do i = 1, len(t)
  if (ichar(t(i:i)) >= ichar('a') .and. ichar(t(i:i)) <= ichar('z')) then
    ! if lowercase, make uppercase
    t(i:i) = char(ichar(t(i:i)) + diff)
  end if
end do
end function
```

Having zeroed in on a prototypical failure point within `dftatom`, in the `utils.f90` file, the next steps are documented in the relevant issue and PR.

The AST representation of the code is :

```
(TranslationUnit [(Module utils [] [(ImplicitNone [])] [(Declaration () [(SimpleAttribute AttrPrivate)] [])]
(Declaration () [(SimpleAttribute AttrPublic)] [(upcase [] [] () None ())]) [(Function upcase [(s) [] [] t ()
[] [] [] [(Declaration (AttrType TypeCharacter [(len () Star)] ()) [(AttrIntent In)] [(s [] [] () None ())])
(Declaration (AttrType TypeCharacter [(()) 20 Value)] ()) [] [(t [] [] () None ())]) [] []])])])
```

No simplifications can be expected to occur from the AST representation to the ASR for this situation so the first order of business is to let the ASR accept the AST representation and pass it through to the backend.

Early on, I considered managing this particular feature in the `cpp` backend, but Ondrej pointed out that the ease-of-use features and guarantees of the `cpp` compiler would lead to a more sloppy ASR implementation.

Conclusions

This third week led me down the rabbit hole with regards to the standard, and a statement comes to mind from Clerman and Spector (2012):

The standard is the contract between the compiler writer and the application developer.

For most of my life I've been the latter, but now, the complex legalese of the standard haunts much of my day.

This attempt at standards level rigor in particular, for `kind` (#373) lead to the discussion on the best way to work with constant expressions at compile time (#375).

My vaccination with the single shot Janssen vaccine and subsequent side effects, have had a deliberating effect. However, the additional meeting with Ondrej more than made up for it in terms of project productivity.

I also satiated mostly my interest in the historical development of “automatic coding” or compiling as we would now call it in the context of Fortran largely due to Cipra ([n.d.](#)), Backus ([1998](#)); and its use at the [University of Iceland](#) [^{fn:2}]. It is very liberating to be able to keep a public record of my reading habits and fancies, however 20 hours of text and code consumed along with ruminative thoughts is rather too much for a single weekly post, might need to eventually step it up. Other interesting things which just haven’t made the cut for this post include finally going through the `gfortran` implementation in some more detail and looking into the linking and loading process via Levine ([2000](#)).

The primary goal for next week shall remain implementing assumed length character arrays and then continuing with the laundry list of `dftatom` tests.

References

Backus, J. 1998. "The History of Fortran I, II, and III." **IEEE Annals of the History of Computing** 20 (4): 68-78. <<https://doi.org/10.1109/85.728232>>.

Cipra, Barry A. n.d. "The Best of the 20th Century: Editors Name Top 10 Algorithms," 2.

Clerman, Norman S., and Walter Spector. 2012. **Modern Fortran: Style and Usage**. New York: Cambridge University Press.

Levine, John R. 2000. **Linkers and Loaders**. San Francisco: Morgan Kaufmann.

expressions](<https://fortran-lang.discourse.group/t/poll-fortran-202x-conditional-expressions-syntax/1425/31>)

records](<https://fortran-lang.discourse.group/t/intel-releases-oneapi-toolkit-free-fortran-2018/471/14?u=rgoswami>)
too